Composition of functional interfaces in Java

Exercise Sheet (Student)

Subject: Technology and Programming Paradigms
Escuela Politécnica de ingeniería (EPI) – University of Oviedo

Block 1. Functional Composition

General Instructions

In the provided skeleton, the method signatures are already created. Your task is to complete the code for each method using the composition operations seen in class.

In all exercises you must:

- Use basic Java and the indicated functional interfaces.
- Show results in console using `System.out.println`.
- Do not modify the method signatures.

Exercise 1. Function: same result with andThen and compose

Complete the method `exercise1_FunctionSameResult()` as follows:

1. Define two mathematical functions:
   - `f(x) = x + 3`
   - `g(x) = 2 * x`
2. Construct:
   - `h1 = f.andThen(g)`
   - `h2 = g.compose(f)`
3. Apply `h1` and `h2` to several values (e.g. 0, 1, 2, 3, 5).
4. Show results in console and comment if both expressions generate the same function.

Exercise 2. Function: compositions with different result

Complete the method `exercise2_FunctionDifferentResult()` as follows:

1. Define:
   - `f(x) = x * x`
   - `g(x) = x + 10`

2. Construct:
   - `h1 = f.andThen(g)`
   - `h2 = g.compose(f)`
   - `h3 = g.andThen(f)`
3. Apply `h1`, `h2` and `h3` to several integer values.
4. Compare the results showing in console which of the functions differs.

Exercise 3. Predicate: checking De Morgan's laws

Complete the method `exercise3_PredicateDeMorgan()` as follows:

1. Define the predicates:
   - `p(x): x > 5`
   - `q(x): x is even`
2. For several integer values (e.g. 3, 4, 6, 9, 10), show:
   - `p(x)` and `q(x)`
   - `!(p(x) && q(x))` and `!p(x) || !q(x)`
   - `!(p(x) || q(x))` and `!p(x) && !q(x)`

Exercise 4. Consumer: effects of order

Complete the method `exercise4_ConsumerOrderEffects()` as follows:

1. Define two consumers:
   - `print`: prints a text with a prefix.
   - `upper`: prints the text in uppercase.
2. Construct:
   - `printThenUpper = print.andThen(upper)`
   - `upperThenPrint = upper.andThen(print)`
3. Apply both consumers to the same string and show the difference of the order in the output.

Exercise 5. Comparator: natural and reversed order

Complete the method `exercise5_ComparatorReversed()` as follows:

1. Create a list of strings and show it.
2. Sort it using `Comparator.naturalOrder()`.
3. Sort it in reverse order using:

```
// Explicit type parameter helps type inference
words.sort(Comparator.<String>naturalOrder().reversed());
```

Exercise 6. Comparator: two criteria

Complete the method `exercise6_ComparatorTwoCriteria()`:

1. Use a `Person` class (name, age) already included in the skeleton.
2. Define comparators `byAge` and `byName`.
3. Construct `byAge.thenComparing(byName)`.
4. Sort the list of people and show it.

Exercise 7. Comparator: nullsLast

Complete the method `exercise7_ComparatorNullsLast()`:

1. Create a list of strings with some null values.
2. Define a comparator by length.
3. Construct `Comparator.nullsLast(byLength)`.
4. Sort the list and show the result.

Block 2. Lazy evaluation with Streams

In this block you will work with `Stream` examples that show that intermediate operations are applied lazily and that terminal operations (`forEach`, `count`, `anyMatch`, `allMatch`, `findFirst`, etc.) can stop processing before traversing all elements.

In the Java project, several empty methods are provided, one per exercise, which you must complete. In all exercises:

- Use simple collections (`List<Integer>`, `List<String>`) and build the `Streams` from them.
- Use `peek` when asked, to see the elements passing through each stage of the `Stream`.
- Write clear messages in console with `System.out.println`.

Exercise 1. Use of peek and filter with limit

In the method corresponding to this exercise you must:

1. Create a list of integers, for example: `[1, 2, 3, 4, 5, 6, 7, 8]`.
2. Build a `Stream` from the list.
3. Chain the following operations:
   - `peek` to show each original value entering the `Stream`.
   - `filter` to keep only even numbers.
   - Another `peek` to show the values that have passed the filter.

- `limit(3)` to keep only the first three even numbers.
- `forEach` to print the final result.

4. Execute the program and observe in what order the messages from `peek`, the filter and the `forEach` appear.

The goal is to see that only the necessary elements are processed to produce the first three final results (lazy evaluation and *limit*).

Exercise 2. Only terminal operations (count, anyMatch, allMatch)

In the method corresponding to this exercise you must:

1. Create a list of integers, for example a list with positive and negative values.
2. Use a `Stream` to:
   - Calculate how many elements are in the list with `count()`.
   - Check if there is any even number with `anyMatch`.
   - Check if all numbers are positive with `allMatch`.
3. Show in console the results of these operations.

The goal is to practice several terminal operations on the same list and understand that each terminal operation traverses the `Stream` again (so it is necessary to create it again from the list).

Exercise 3. Chaining multiple intermediate operations

In the method corresponding to this exercise you must:

1. Create a list of strings with repeated words, mixing uppercase and lowercase.
2. Build a `Stream` from the list and chain:
   - `filter` to keep only the words whose length is greater than or equal to a certain number (e.g. 4).
   - `distinct()` to remove duplicates.
   - `sorted()` to sort alphabetically.
3. Terminate the pipeline with `forEach` to print the resulting words.

The goal is to see how several intermediate operations are combined before a terminal operation.

Exercise 4. anyMatch, allMatch and noneMatch with "short-circuit"

In the method corresponding to this exercise you must:

1. Create a list of integers.
2. Build a `Stream` and use `peek` to show the values being processed.
3. Use `anyMatch` to check if there is any number that meets a condition (e.g., being greater than a certain prefixed value).
4. Re-create the `Stream` and use `allMatch` to check if all numbers meet another condition.
5. Re-create the `Stream` and use `noneMatch` to check that no number meets a specific condition.

You can include messages in the `filter` lambdas or in `peek` to observe that these operations stop as soon as they know the result (what we call "short-circuit").

Exercise 5. limit and skip on Streams

In the method corresponding to this exercise you must:

1. Create a list of integers from 1 to at least 10.
2. Build a `Stream` and apply:
   - `skip(3)` to skip the first three numbers.
   - `limit(4)` to keep only the next four.
3. Use `forEach` to show in console the final numbers.

The goal is to practice the combination of `skip` and `limit` to work only with a window of elements of the `Stream`.

Exercise 6. findFirst and lazy search

In the method corresponding to this exercise you must:

1. Create a list of words of different lengths.
2. Build a `Stream` and chain:
   - `peek` to show each word entering the `Stream`.
   - `filter` to keep only the words whose length is greater than a certain value (e.g., greater than 8).
   - `findFirst()` as terminal operation to obtain, if it exists, the first word that meets the condition.

The goal is to observe that, thanks to lazy evaluation, only the necessary words are processed until finding the first one that meets the condition.

Block 3. Mini-app: Truth Table Lab

In this block you will work with a console mini-app that builds truth tables for several propositional formulas using:

- Functional interfaces `Predicate<boolean[]>` to represent formulas.
- Composition of predicates with `and()`, `or()` and `negate()`.
- `Streams` to classify formulas as tautology, contradiction or contingent.

In the provided Java skeleton you already have:

- A constant indicating the number of propositional variables and an array of names (e.g. `"p"`, `"q"`, `"r"`).
- An auxiliary class `NamedFormula`, which stores the name of a formula and the corresponding predicate.
- A method `decode` that converts an integer into a `boolean[]` array with the truth values of the variables.
- A method `runLab()`, which shows a header, calls the methods of this block and shows on screen the classification of each formula.

Your task is to complete the functional methods marked as `// TODO` in the skeleton:

- `generateInterpretations(int n)`
- `buildFormulas(...)`
- `classifyFormula(...)`

Below is what is requested in each exercise.

Exercise 1. Generate all interpretations

Complete the method `generateInterpretations(int n)` so that it returns a list with all possible interpretations for `n` propositional variables.

- Each interpretation will be represented as a `boolean[]` array of length `n`. Position `0` corresponds to the first variable, `1` to the second, and so on.
- As you know there are 2^n possible interpretations for propositional formulas of n variables. You can calculate this number in the method itself.
- The proposed algorithm must generate those 2^n integers in order, from 0 to $2^n - 1$
- You must use the `decode` method already implemented in the skeleton to convert each integer into a `boolean[]` interpretation.
- You must use a `Stream` to generate the integers from `0` to the last necessary value and, for each integer, add the corresponding interpretation to a list.

Upon completion, `generateInterpretations` must return a `List<boolean[]>` with all interpretations.

Exercise 2. Build formulas by predicate composition

In the method `buildFormulas` you are provided with three predicates of type `Predicate<boolean[]>` representing the propositional variables:

- A predicate for the first variable (e.g., `"p"`).
- A predicate for the second variable (e.g., `"q"`).
- A predicate for the third variable (e.g., `"r"`).

Your task is to build a list of `NamedFormula` objects using these predicates and the composition methods of `Predicate`:

- `and()`
- `or()`
- `negate()`

You must add at least the following formulas (names are indicative; you can use similar names if the Java skeleton indicates so):

1. A formula equivalent to `p AND q`.
2. A formula equivalent to `p OR NOT p`, which should be a tautology.
3. A formula equivalent to `p AND NOT p`, which should be a contradiction.
4. A formula equivalent to `(p AND q) implies r`, which you can define as `NOT(p AND q) OR r`.
5. A formula equivalent to `p implies (q OR r)`, which you can define as `NOT p OR (q OR r)`.

In each case:

- Create the corresponding predicate combining the variable predicates with `and()`, `or()` and `negate()`.
- Create a new `NamedFormula` object with the formula name and the resulting predicate.
- Add the formula to the list that `buildFormulas` will return.

The method must return a `List<NamedFormula>` with all defined formulas.

Exercise 3. Classify formulas with Streams

In the method `classifyFormula` you must determine if a formula is:

- **Tautology**: true in all interpretations.
- **Contradiction**: false in all interpretations.
- **Contingent**: true in some interpretations and false in others.

In the skeleton you have:

- The formula predicate, accessible from the `NamedFormula` object.
- The list of interpretations, where each `boolean[]` represents an assignment of truth values for the variables.
- An enumerated type `FormulaType` with the possible values `TAUTOLOGY`, `CONTRADICTION` and `CONTINGENT`.

Your task is:

1. Use `interpretations.stream().allMatch(...)` to check if the formula is true in all interpretations.
2. Use `interpretations.stream().noneMatch(...)` to check if the formula is false in all interpretations.
3. Return `FormulaType.TAUTOLOGY` if all interpretations satisfy the formula.
4. Return `FormulaType.CONTRADICCTION` if no interpretation satisfies it.
5. In any other case, return `FormulaType.CONTINGENT`.

Remember that `allMatch` and `noneMatch` are terminal operations that stop as soon as they find an interpretation that breaks the condition (lazy evaluation).

When executing `runLab()`, these methods will be called and the name of each formula along with its classification will be shown on screen.